CSC-239
Java Programming:
Chapter 7: Single-dimensional Arrays (Part 1)

September 24, 2025

NOTICE to Students: Portions of these lecture slides are
Copyright 2021 Pearson Education, Inc.
We have a license to use this material ***only*** in the context of this course.
There is NO PERMISSION to share these lecture slides with anyone not taking the course.
There is NO PERMISSION to post these lecture slides on the Internet.

Reminder: Submitting Files to *Moodle*

- When you submit your source code (*.java) files to *Moodle*, you MUST follow the **Submit Assignment to *Moodle*** procedure *completely*: be sure that the final Submission Status is:

  "Submitted for grading".



- This procedure is described in detail in the **Submit_Assignment_to_Moodle.pdf** document, which is available on *Moodle*, under the "**Handouts**" topic.

Online Information

- Java Tutorials:

  https://docs.oracle.com/javase/tutorial/

- Java API:

  https://docs.oracle.com/en/java/javase/21/docs/api/index.html

- The official "*Grader Than*" documentation:

  https://docs.graderthan.com/student/

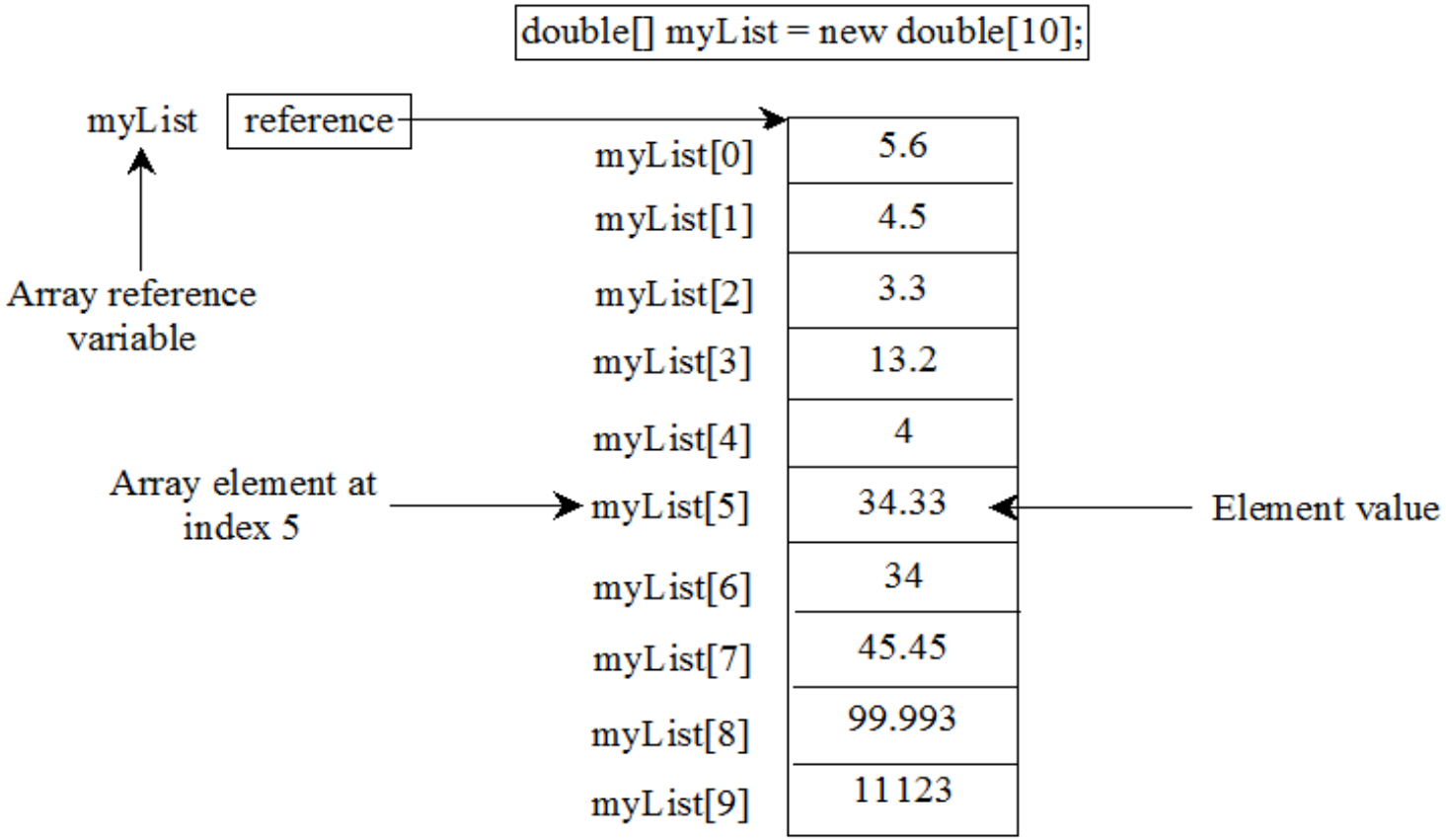
- *Grader Than* Support E-mail:

  support@graderthan.com

What is an Array?

An array is a container object that holds a number of values of a single type.

[http://docs.oracle.com/javase/tutorial/java/nutsandbolts/arrays.html]



Declaring Array Variables

- Syntax for an array reference variable:

  datatype[] *arrayRefVar*;

  Example:

```
double[] myList;
```

- An array reference variable is *not* an array.
- Alternative syntax:

  datatype *arrayRefVar*[]; // not preferred

  Example:

```
double myList[];
```

Creating Arrays

- Creating an array is similar to creating any object:

arrayRefVar = new *datatype*[*arraySize*];

Example:

```
myList = new double[10];
```

- You can declare and create in one step:

datatype[] *arrayRefVar* = new *datatype*[*arraySize*];

Example:

```
double[] myList = new double[10];
```

Accessing Array Elements

- Individual elements of the array are accessed by their index.
  - Array indices are *0-based:* from 0 to *arraySize*-1.

Example:

```
double[] myList = new double[10];
```

`myList[0]` references first element

`myList[9]` references last element

The Length of an Array

- Once an array is created, its size is fixed. It cannot be changed. You can find its size using

  arrayRefVar.length

  Referring to the example from the previous slide:

  `myList.length` evaluates to 10

- The length value *appears* to be a member variable of the array object:
  - public access
  - read-only (cannot be modified)

Default Values

When an array is created, its elements are assigned the default value of

0 for the numeric primitive data types,

'\u0000' for char types, and

false for boolean types.

Using Indexed Variables

- After an array is created, an indexed variable can be used in the same way as a regular variable.
  - The following example adds values in myList[0] and myList[1], and places the result in myList[2]:

```
myList[2] = myList[0] + myList[1];
```

Shorthand Notation for Declaring, Creating, Initializing

```
double[] myList = {1.9, 2.9, 3.4, 3.5};
```

This shorthand notation is equivalent to the following statements:

```
double[] myList = new double[4];

myList[0] = 1.9;

myList[1] = 2.9;

myList[2] = 3.4;

myList[3] = 3.5;
```

CAUTION

- With the shorthand notation, you must declare, create, and initialize the array all in one statement. Splitting it would cause a syntax error.
- For example, the following is wrong:

```
double[] myList;
myList = {1.9, 2.9, 3.4, 3.5};  // error
```

Populating an array with user input

```
double[] myList = new double[10];

java.util.Scanner input
    = new java.util.Scanner(System.in);

System.out.print("Enter " + myList.length + "
   values: ");

for (int i = 0; i < myList.length; i++) {
    myList[i] = input.nextDouble();
}
```

Printing arrays

```
for (int i = 0; i < myList.length; i++) {
  System.out.print(myList[i] + " ");
}
```

Summing all elements

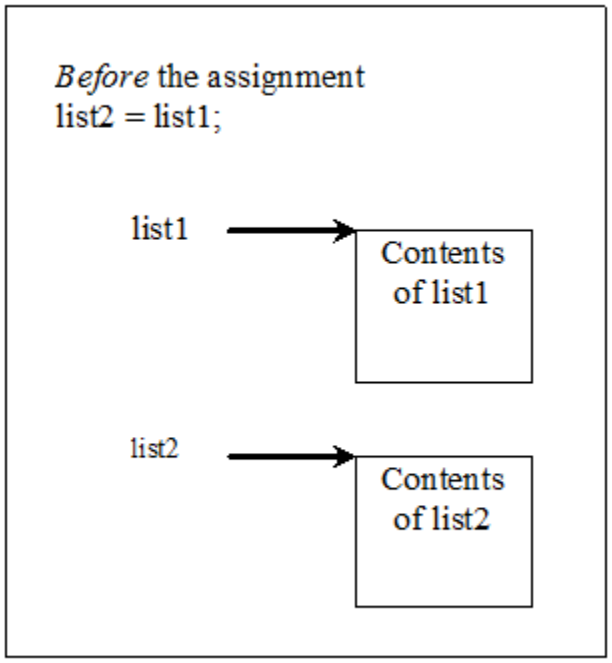
```
double total = 0;

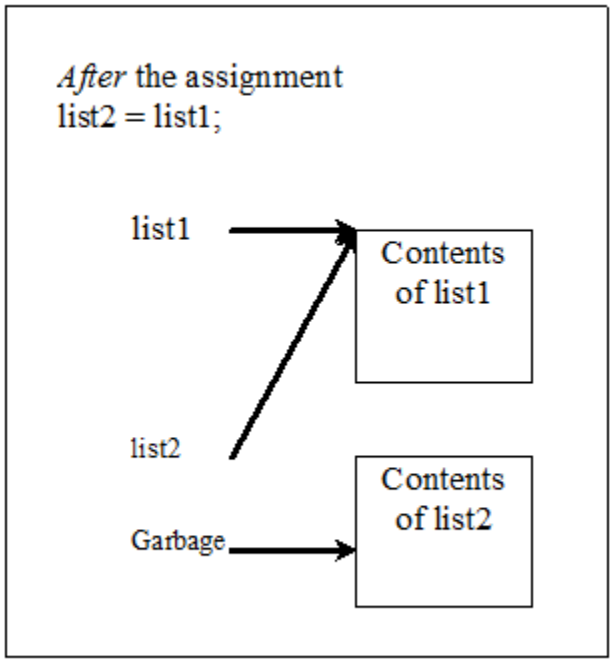
for(int i = 0; i < myList.length; i++) {
     total += myList[i];
}
```

Copying Arrays (incorrect)

- You might *attempt* to copy an array this way:

```
list2 = list1;    // Not a REAL copy
```





Copying Arrays (correct)

Use a loop:

```
int[] sourceArray = {2, 3, 1, 5, 10};
int[] targetArray = new
     int[sourceArray.length];

for (int i = 0;i < sourceArray.length; i++){
     targetArray[i] = sourceArray[i];
}
```

Passing Arrays to Methods

- Java uses *pass by value* to pass arguments to a method.
- For a parameter of a primitive type value, the actual value is passed.
  - Changing the value of the local parameter inside the method does not affect the value of the variable outside the method.
- For a parameter of an array type, the *value* passed to the method is a *reference* to the array.
  - Any changes to the array that occur inside the method body will affect the original array that was passed as the argument.

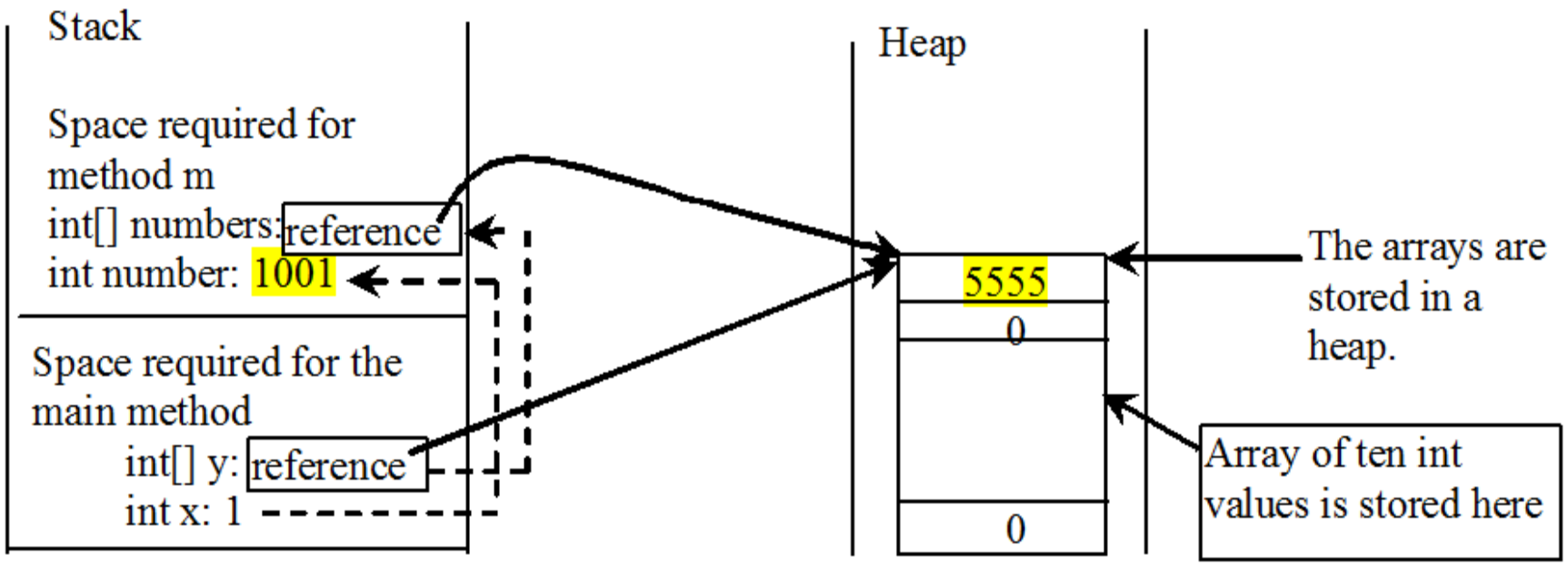
(See also Section 7.6 of the textbook.)

Example: Pass Array `y` to method `m`

```
public class TestArrayArguments {
  public static void main(String[] args) {
      int x = 1; // x represents an int value
      int[] y = new int[10]; // array of int values

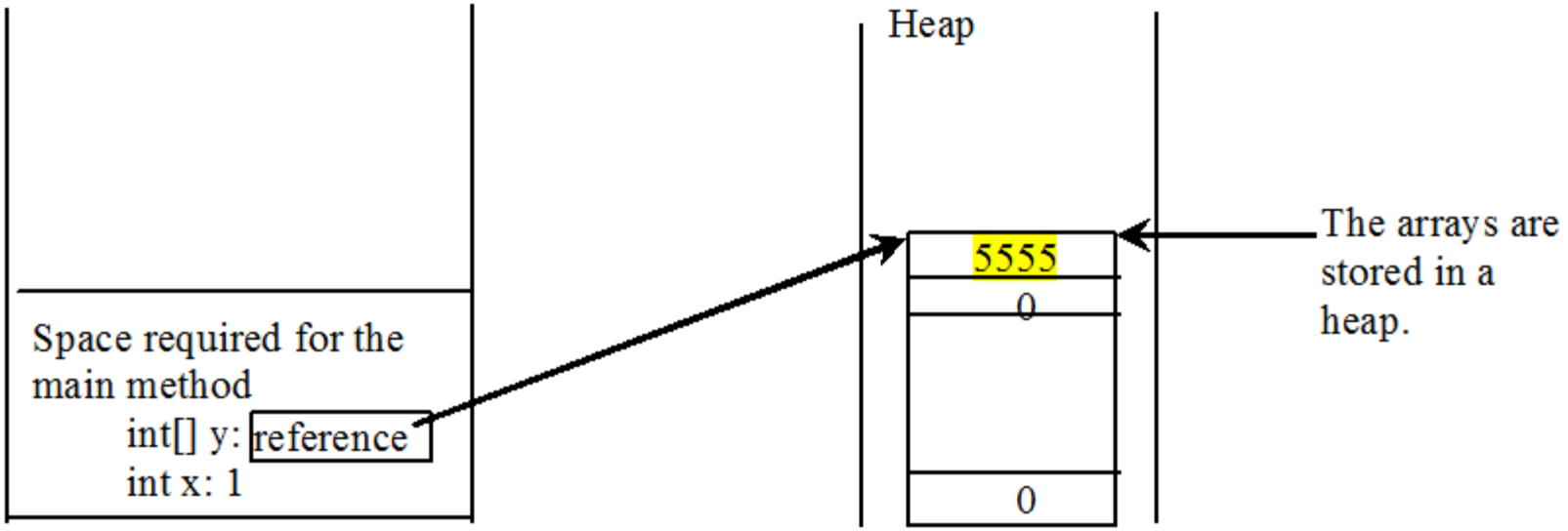
      m(x, y); // Invoke m with arguments x and y
      System.out.println("x is " + x);
      System.out.println("y[0] is " + y[0]);
  } // end main
// Modify number and numbers[0]:
  public static void m(int number, int[] numbers) {
      number = 1001;      // Modify number
      numbers[0] = 5555; // Modify numbers[0]
  } // end m
} // end TestArrayArguments
```

Call Stack vs. The "Heap"



- When invoking m(x, y), the *values* of x and y are passed to number and numbers.
- Because the value of y is a reference value to the array, numbers now contains the same reference value to the *same* array.

Heap: Dynamic Memory Allocation



- The JVM stores the array in an area of memory, called the *heap*
  - Used for dynamic memory allocation
  - Blocks of memory are allocated and freed in an arbitrary order.

Anonymous Array

The statement

```
printArray(new int[]{3, 1, 2, 6, 4, 2});
```

creates an array using the following syntax:

new *dataType*[]{*literal0*, *literal1*, ..., *literalK*};

There is no explicit reference variable for the array. Such array is called an *anonymous array*.

Returning an Array from a Method

```
public static int[] reverse(int[] list) {
  int[] result = new int[list.length];

  for (int i = 0, j = result.length - 1;
           i < list.length;
             i++, j--)
  {
    result[j] = list[i];
  }
  return result;
}
```

Enhanced for Loop (for-each loop)

In some circumstances you may use a different variation of the **for** loop that enables you to traverse the complete array sequentially without using an index variable. For example, the following code displays all elements in the array myList:

```
for (double value: myList)
   System.out.println(value);
```

The general syntax is:

```
for (type variableName: arrayName) {
   statement(s)
}
```

You still have to use an index variable if you wish to traverse the array in a different order or change the elements in the array.

Arrays of Strings

- Arrays are *not* limited to numeric data types.
- The **String** class includes a very convenient method that inputs a long string and outputs an array of shorter strings.
- The original string is not modified, but its contents are copied into the new array of **String** objects.

The `split` Method

- The **`split`** method (in the **String** class) scans the contents of a **String** object for matches to an input parameter. The **`split`** method copies the original text into an array of **String** objects around the matches to the input.
- The search string is called a *regular expression,* and can be as simple as a single character, or it can be very elaborate.
- Very often a simple single-character search pattern is called a "delimiter" character.

A very simple `split` example

```
public static void main(String[] args) {
 String sentence = "We the People of "
  +"the United States, "
  +"in Order to form a more perfect Union, "
        …
  +"the United States of America.";
  String[] words = sentence.split(" ");
  for (int i = 0; i < words.length; i++){
    System.out.printf("i=%d, text=%s%n",
                  i, words[i]);
  }
}
```

Initializing arrays with random values

```
for (int i = 0; i < myList.length; i++) {
  myList[i] = Math.random() * 100;
}
```

Finding the largest element

```
double max = myList[0];
for (int i = 1; i < myList.length; i++) {
  if (myList[i] > max) max = myList[i];
}
```

The `arraycopy` Utility

Copy **length** elements from the **sourceArray** object, beginning at index **src_pos**, into the **targetArray**, beginning at index **target_pos**.

```
arraycopy (sourceArray, src_pos, targetArray,
   target_pos, length) ;
```

Example:

```
System.arraycopy(sourceArray, 0,
 targetArray, 0, sourceArray.length);
```